



UNIVERSIDADE FEDERAL DA FRONTEIRA SUL
CIÊNCIA DA COMPUTAÇÃO - Campus Chapecó

Alunos: Jean Claude Rossa e Lucas Smaniotto Schuch

PROJETO 2: Construção de Analisador Sintático

Resumo

Este trabalho apresenta a implementação da primeira etapa de um analisador sintático para uma linguagem formal simplificada, desenvolvida no contexto da disciplina de Construção de Compiladores. O foco concentrou-se na definição e no tratamento da gramática livre de contexto, incluindo a eliminação de símbolos inúteis, a fatoração da gramática, o cálculo dos conjuntos FIRST e FOLLOW, a construção da coleção canônica de itens LR(0), a geração da tabela de parsing SLR(1) e a implementação do algoritmo de reconhecimento sintático. Como suporte às etapas futuras, também foi incluída uma tabela de símbolos básica, capaz de registrar informações iniciais sobre os tokens reconhecidos. Os resultados obtidos mostram que a solução construída realiza corretamente a análise sintática de cadeias válidas e emite mensagens de erro para entradas inválidas, atendendo aos requisitos previstos para a Etapa 1 do projeto.

Introdução

Os reconhecedores sintáticos constituem uma etapa central no processo de compilação, sendo responsáveis por verificar se a sequência de tokens produzida pelo analisador léxico obedece às regras estruturais definidas pela linguagem. Em termos práticos, essa fase permite identificar se um programa ou comando foi escrito de acordo com a gramática especificada, servindo como base para etapas posteriores, como análise semântica, geração de código intermediário e otimização.

A aplicação dos reconhecedores sintáticos entre suas principais características, destacam-se o uso de gramáticas livres de contexto para modelar a linguagem, a construção de mecanismos automáticos de decisão sintática e a capacidade de detectar e relatar erros estruturais na entrada.

Neste trabalho, o problema proposto consistiu em desenvolver a infraestrutura sintática inicial de um compilador para uma linguagem simplificada, recebendo como entrada a fita

produzida pelo reconhecedor léxico e gerando como saída a aceitação da cadeia ou mensagens de erro sintático. O objetivo principal foi implementar apenas a Etapa 1 do projeto, isto é, a construção completa do reconhecedor sintático com base em uma tabela SLR(1), além de preparar uma tabela de símbolos inicial para apoiar as próximas fases do compilador.

Referencial teórico

O desenvolvimento de um analisador sintático parte do uso de uma gramática livre de contexto (GLC), formalismo que descreve a estrutura sintática de uma linguagem por meio de produções, símbolos terminais, símbolos não terminais e um símbolo inicial. A partir dessa gramática, é possível modelar as construções válidas da linguagem e estabelecer as regras que o parser deve reconhecer.

Antes da construção do analisador, é comum aplicar transformações na gramática para torná-la mais adequada ao processamento automático. Entre essas transformações está a eliminação de símbolos inúteis, que remove símbolos improdutivos, isto é, incapazes de gerar cadeias terminais, e símbolos inalcançáveis, que não podem ser atingidos a partir do símbolo inicial. Outra técnica importante é a fatoração, utilizada para eliminar prefixos comuns entre produções e reduzir ambiguidades operacionais na análise.

Na sequência, calculam-se os conjuntos FIRST e FOLLOW. O conjunto FIRST de um símbolo ou sequência indica quais terminais podem aparecer no início de uma derivação. Já o conjunto FOLLOW informa quais terminais podem surgir imediatamente após um determinado não terminal em alguma derivação válida. Esses conjuntos são fundamentais para a construção de tabelas de parsing e para a definição de ações de redução em analisadores bottom-up.

O método adotado neste trabalho foi o parsing SLR(1), uma técnica de análise sintática ascendente baseada em reduções e deslocamentos. Esse tipo de parser utiliza a coleção canônica de itens LR(0) para representar estados do reconhecimento e, a partir das transições entre esses estados e dos conjuntos FOLLOW, constrói a tabela de parsing. Durante a execução, o algoritmo mantém uma pilha de estados e símbolos, consultando a tabela para decidir se deve realizar uma ação de shift, reduce, accept ou sinalizar erro.

Por fim, a tabela de símbolos é uma estrutura fundamental em compiladores, pois armazena informações sobre os elementos identificados durante a análise, como identificadores, palavras reservadas, tipos e valores. Mesmo em uma implementação inicial,

essa tabela já representa um passo importante para o suporte às fases semânticas e de geração de código.

Implementação e resultados

A implementação desenvolvida concentrou-se exclusivamente na Etapa 1 do projeto. Inicialmente, foi definida a gramática da linguagem considerada, cujo núcleo sintático descreve comandos iniciados pela palavra reservada `se`, seguidos por um identificador, a palavra `foi` e um bloco, com possibilidade de extensão pela construção seguida de outro bloco. Como parte da validação conceitual do processo, a gramática original foi propositalmente criada com símbolos inúteis, permitindo demonstrar a aplicação prática dos algoritmos de eliminação de improdutivos e inalcançáveis.

Após essa etapa de limpeza, foi aplicada a fatoração da gramática, especialmente para resolver o prefixo comum presente em produções do não terminal responsável pelos comandos. Essa transformação tornou a especificação mais adequada à construção da tabela de parsing e evidenciou uma preocupação metodológica com a qualidade formal da gramática utilizada.

Na etapa seguinte, foram calculados os conjuntos FIRST e FOLLOW para todos os não terminais relevantes. Esses conjuntos serviram de base para a construção da coleção canônica de itens LR(0), que representa os estados possíveis do analisador sintático. Com os estados e transições definidos, foi então construída a tabela SLR(1), contendo as ações de deslocamento, redução, aceitação e os desvios entre estados para não terminais.

Com a tabela pronta, foi implementado o algoritmo de reconhecimento sintático do tipo shift-reduce. Esse algoritmo recebe como entrada a fita produzida pelo reconhecedor léxico, codificada por rótulos de tokens, converte esses rótulos para os terminais da gramática e executa a análise com base na pilha e na tabela ACTION/GOTO. O resultado da execução pode ser a aceitação da cadeia ou a emissão de uma mensagem de erro sintático, indicando a posição do problema, o símbolo encontrado e o conjunto de símbolos esperados naquele contexto. Para evidenciar esse comportamento, pode-se observar diretamente a execução da função de reconhecimento sobre uma cadeia válida e outra inválida. No código principal, a validação é feita por chamadas como as seguintes:

```
Python
parser = construir_parser()
```

```
reconhecer('3 1 9 1 6 1 $', parser) # se id foi id sai id
reconhecer('3 9 1 $', parser)       # se foi id
```

No primeiro caso, a fita é reconhecida corretamente. O analisador percorre a pilha com ações de shift e reduce até alcançar a aceitação da cadeia, além de preencher a tabela de símbolos com os tokens identificados. Um trecho representativo da saída é mostrado a seguir:

```
None
FITA de entrada : 3 1 9 1 6 1 $
Terminais      : se id foi id sai id $

PILHA          ENTRADA          ACAO
-----
?              se id foi id sai id $  empilha 'se' -> estado 1
se              id foi id sai id $    empilha 'id' -> estado 5
...
Programa       $                    ACEITA

RESULTADO: cadeia ACEITA
```

Já no segundo caso, a entrada é rejeitada porque, após o terminal “se”, o parser esperava um identificador, mas encontrou o terminal “foi”. A mensagem emitida pelo reconhecedor evidencia o ponto e a natureza do erro:

```
None
FITA de entrada : 3 9 1 $
Terminais      : se foi id $

PILHA          ENTRADA          ACAO
-----
?              se foi id $      empilha 'se' -> estado 1
se              foi id $        ERRO

RESULTADO: erro sintático na posição 2: encontrado 'foi', esperado um de
['id']
```

Além do reconhecimento sintático, foi implementada uma tabela de símbolos básica. Nessa estrutura, cada token válido é registrado com informações iniciais como ordem de ocorrência, lexema categorizado, classe léxica e campos reservados para tipo e valor. Embora

esses últimos ainda permaneçam sem preenchimento semântico nesta etapa, sua presença já prepara a base necessária para o avanço do projeto nas fases seguintes.

Como estudo de caso para validação, o sistema permite testar tanto cadeias válidas quanto inválidas. Cadeias compatíveis com a gramática são reconhecidas e aceitas, enquanto entradas malformadas resultam em mensagens de erro sintático. Um exemplo de validação negativa é a cadeia em que falta o identificador após a palavra reservada `se`, situação em que o parser detecta a inconsistência e informa a falha. Esse comportamento demonstra que a implementação atende ao objetivo da etapa: reconhecer estruturas corretas da linguagem e rejeitar aquelas que não obedecem à gramática definida.

É importante destacar que as Etapas 2, 3 e 4 não foram implementadas neste trabalho. Portanto, ainda não há ações semânticas efetivas, análise semântica, geração de código intermediário ou otimização. A implementação atual corresponde somente à infraestrutura sintática e à tabela de símbolos inicial exigidos na primeira fase do projeto.

Conclusões

O trabalho realizado cumpriu o objetivo proposto para a Etapa 1 do projeto, ao desenvolver um reconhecedor sintático baseado em parsing SLR(1) e uma tabela de símbolos inicial. Entre as principais dificuldades do desenvolvimento, destacam-se a formalização correta da gramática, a adequação da especificação para a construção da tabela SLR(1) e a implementação consistente do mecanismo de redução e deslocamento. Mesmo em uma linguagem simplificada, essas etapas exigem atenção à coerência formal entre gramática, estados, transições e decisões do parser.

Os resultados finais foram satisfatórios para a proposta da primeira etapa, pois a solução consegue distinguir cadeias válidas de inválidas e apresentar mensagens de erro sintático, além de manter uma estrutura inicial de tabela de símbolos para uso futuro. Como perspectiva de continuidade, o trabalho pode ser expandido com a implementação da análise semântica, preenchendo a tabela de símbolos com informações de tipo, escopo ou valor, seguida da geração de código intermediário e da aplicação de técnicas de otimização.